

HelixGitpx

HelixGitpx

**Eine einzige Quelle der Wahrheit, überall gespiegelt -
förderiertes Git über ein Dutzend Hosts.**

Zusammenfassung

HelixGitpx (Helix Git Proxy eXtended) ist ein förderierter Git-Proxy, der eine einzige Quelle der Wahrheit über mehrere Git-Hosts hinweg spiegelt und unvermeidliche Konflikte mit richtlinien- und AI-gestützten Abläufen auflöst. Das Projekt hat die Version 1.0.0 GA erreicht.

Kurzbeschreibung

HelixGitpx ist ein förderierter Git-Proxy, der eine einzige Quelle der Wahrheit über ein Dutzend oder mehr Git-Hosts – darunter GitHub, GitLab, Gitea, Bitbucket, Azure DevOps, AWS CodeCommit und weitere – spiegelt und Synchronisationskonflikte mit richtlinien- und AI-gestützten Abläufen löst. Veröffentlicht als Version 1.0.0 GA.

Ausführliche Beschreibung

HelixGitpx – „Helix Git Proxy eXtended“ – ist ein förderierter Git-Proxy, der eine einzige Quelle der Wahrheit über mehrere Git-Hosts hinweg spiegelt und die Konflikte auflöst, die unweigerlich entstehen, sobald dasselbe Repository an mehr als einem Ort existiert. Die unterstützten Upstream-Hosts lesen sich wie eine Landkarte des gesamten Git-Ökosystems: GitHub, GitLab, GitFlic, GitVerse, Gitea, Gitee, Bitbucket, Azure DevOps, AWS CodeCommit, Forgejo, SourceHut sowie generisches Git-over-HTTPS. Während ein naives `git push` zu einem Dutzend Remotes entweder scheitern oder – schlimmer noch – die Spiegel stillschweigend auseinanderdriften lassen würde, greift

HelixGitpx mit richtlinien- und AI-gestützten Auflösungsabläufen ein, die Abweichungen wieder zu einer einzigen verbindlichen Wahrheit zusammenführen.

Das Projekt liegt in Version 1.0.0 GA vor, mit Meilensteinen von m1-foundation bis m8-ga, die einen vollständig durchgeplanten Weg von der Grundlage bis zur allgemeinen Verfügbarkeit markieren. Es ist als dreistufiges Produkt konzipiert: ein Go-Monorepo (eine Plattform mit achtzehn Diensten, Codegenerierung und Scaffolding-Tools), das den Föderationskern trägt; eine Angular-Webanwendung mit 19 + Nx; sowie Kotlin-Multiplattform- und Compose-Clients, die native Erlebnisse für Android, iOS und Desktop aus gemeinsamem Code bereitstellen. Die Plattformausslieferung ist durch und durch Kubernetes-nativ – mit Helm-Charts, Argo-CD-Anwendungen, Kustomize-Overlays, SQL und OPA-Richtlinien – und CI-Pipelines sind durch explizite Auslösung abgesichert, sodass nichts versehentlich ausgerollt wird. Die öffentliche Dokumentation wird als Docusaurus-Seite (docs.helixgitpx.io) bereitgestellt, begleitet von einer Astro-Marketingseite (helixgitpx.io).

Die Governance ist streng und verfassungsbasiert – und das ist ein Feature, kein bloßer Formalismus: Das tragende Dokument ist das Projekt-Constitution, dessen Artikel II eine siebenstufige Testmatrix mit 100% Abdeckung *pro Typ und pro geändertem Modul* vorschreibt, wobei Mocks nur in Unit-Tests erlaubt sind und kein einziger Test übersprungen werden darf. Ein einmaliger Verifizierer führt jede Artefaktprüfung sowie `go vet` und `go test` im gesamten Workspace aus, und jeder Push wird an alle konfigurierten Upstreams weitergeleitet – sodass „die Spiegel synchron sind“ etwas ist, das das System bei jedem Commit erzwingt, statt dass ein Mensch sich daran erinnern müsste.

Warum wir es entwickelt haben

Ein Repository über viele Git-Hosts hinweg authentisch zu halten – sei es aus Redundanzgründen, Souveränitätsbedenken oder um regionale Plattformen zu erreichen – ist fehleranfällig und manuell aufwendig, und auseinanderdriftende Spiegel sind schwer wieder in Einklang zu bringen. HelixGitpx wurde entwickelt, um das Spiegeln auf mehrere Upstreams zu einer erstklassigen, konfliktbewussten Fähigkeit zu machen.

Warum es ein Game-Changer ist

Es verwandelt „Push zu vielen Remotes und hoffen“ – den fragilen, manuellen Status quo – in eine kontrollierte Föderation mit einer einzigen verbindlichen Quelle der Wahrheit und automatisierter, richtlinien- und AI-gestützter Konfliktlösung. Und

das über ein ungewöhnlich breites Spektrum an Hosts hinweg, das gezielt auch regionale Plattformen (GitFlic, GitVerse, Gitee) einbezieht, die die meisten Tools stillschweigend ignorieren. So werden Redundanz, Datensouveränität und die Anbindung an diese Ökosysteme nicht länger zum Wartungsaufwand, sondern zu einer einzigen Fähigkeit, die man einmal konfiguriert.

Was innovativ ist

- **Breite der Upstreams** – über ein Dutzend Git-Hosts, von GitHub und GitLab bis hin zu regionalen Plattformen wie GitFlic, GitVerse und Gitee, alle hinter einem einzigen Proxy vereinheitlicht.
- **Richtlinien- und AI-gestützte Konfliktlösung** – Abweichungen werden durch eine Policy-Engine und einen AI-Resolver abgeglichen, nicht durch manuelles Vergleichen von Spiegeln.
- **Föderation mit einer einzigen Quelle der Wahrheit** – ein Push-Modell für alle Upstreams, bei dem ein autoritatives Repository die Wahrheit darstellt und jeder Host ein synchron gehaltener Spiegel ist.
- **Streng durchgesetzte, verfassungsbasierte Tests** – sieben Testtypen mit 100 % Abdeckung pro Typ, ohne Ausnahmen, nachgewiesen durch ein One-Shot-Green-Suite-Skript statt auf Vertrauensbasis.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Abweichungen und Konflikte über viele Upstreams hinweg.** Dasselbe Repository, das an einem Dutzend Orten existiert, driftet sofort, sobald zwei Hosts unterschiedliche Änderungen akzeptieren. Gelöst durch richtlinien- und AI-gestützte Auflösungsprozesse, die an einer einzigen Quelle der Wahrheit verankert sind, sowie einen synchronisierten Push für alle Upstreams, der dafür sorgt, dass alle Spiegel auf diese eine Wahrheit konvergieren.
- **Einheitliche Unterstützung heterogener Git-Hosts.** Jeder Host hat seine eigene Authentifizierung, Eigenheiten und API. Gelöst durch pro-Upstream-Konfigurationsskripte im Verzeichnis upstreams/ und eine Plattformschicht, die diese Unterschiede abstrahiert, sodass das Hinzufügen eines neuen Hosts eine Frage der Konfiguration ist, nicht der Neuentwicklung.
- **Korrektheit vor jedem Merge nachweisen.** Gelöst durch eine verpflichtende Testmatrix mit sieben Testtypen und ein verify-everything.sh-One-Shot-Gate, das die vollständige Prüfung durchläuft – und bei Nichterreichbarkeit eines

Clusters sauber abbricht, sodass die Korrektheit sowohl lokal als auch in der CI nachweisbar ist.

Technologie-Stack

- **Go-Monorepo** – der Kern-Proxy und Föderationsmotor: eine Plattform mit 18 Diensten, Codegenerierung und Scaffolding, alles in einem Repository, damit die gesamte Engine als Einheit gebaut und getestet wird.
- **Angular 19 + Nx** – die Webanwendung, wobei Nx die Monorepo-Build- und Caching-Struktur bereitstellt, die eine große Frontend-Anwendung benötigt.
- **Kotlin Multiplatform + Compose** – native Android-, iOS- und Desktop-Client-Shells, die aus einer gemeinsamen Codebasis generiert werden, sodass drei Plattformen nicht drei separate Implementierungen bedeuten.
- **Kubernetes + Helm + Argo CD + Kustomize** – cloud-native Bereitstellung: Helm verpackt die Veröffentlichung, Kustomize-Overlays passen sie an die jeweilige Umgebung an, und Argo CD gleicht sie via GitOps ab, sodass der Cluster-Zustand mit Git übereinstimmt.
- **OPA (Rego)** – Richtlinien als Code für Konfliktlösung und Zugriffskontrolle, wodurch Autorisierungsentscheidungen deklarativ und auditierbar bleiben.
- **Docusaurus** – die öffentliche Dokumentationsseite (docs.helixgitpx.io); **Astro** – die Marketing-Website (helixgitpx.io), wobei jedes Tool auf seinen Inhaltstyp abgestimmt ist.
- **mise** – eine festgelegte, reproduzierbare Toolchain, sodass jeder Mitwirkende und CI-Runner exakt dieselben Versionen verwendet.

Status- und Ehrlichkeitshinweise

- **Status: ausgeliefert.** Die README des Projekts deklariert v1.0.0 GA mit den Meilensteinen m1-foundation bis m8-ga als getaggt. („v1.0.0 GA“ ist die Eigenangabe des Projekts in der README.)
- **Lizenz: noch zu klären.** Die Berichte GitHub und API nennen MIT, während der Lizenzabschnitt der README Apache-2.0 (Code) / CC-BY-SA-4.0 (Dokumentation) angibt – vor der Veröffentlichung ist dies anhand der tatsächlichen LICENSE-Datei zu klären.
- Die Dokumentations- (docs.helixgitpx.io) und Marketing-URLs (helixgitpx.io) entsprechen den Angaben in der README und wurden nicht eigenständig abgerufen – Live-Status UNBESTÄTIGT.

Prioritätsstufe: Helix-primär.